

When equal exponents are not equal keys

An incremental audit of Asymptotic 1.8.0

New correctness evidence, a tested semantic-merge repair,
real-domain enforcement, numerical diagnostics,
and comparison with Wolfram Language

Prepared for	Vladimir Reshetnikov
Repository	https://github.com/VladimirReshetnikov/Asymptotic
Audited snapshot	921387e5ba1239bfda96e63e64e89bf63d9c41e6
Native evaluator	Wolfram Language 15.0.0, Linux x86 (64-bit), build dated 6 May 2026
Audit date	9 September 2026

Scope of the contribution. This is a delta to the nine existing reviews, not another presentation of their backlog. Its principal new finding is a native-reproduced false remainder caused by structural grouping of mathematically equal transcendental exponents. The report also turns the earlier real-coefficient concern into public counterexamples and gives an exact oracle for a numerical diagnostic that cannot resolve the error it displays.

Executive assessment

The package has substantial value, but a shared sparse-support invariant is currently false. The strongest new result of this audit is not an absent feature or a speculative optimization. For two provably equal transcendental weights, ordinary `jetMerge` can retain two separate rows. Downstream code assumes the rows have distinct increasing weights. That mismatch can produce an invalid big-O bound, return no terms for a one-term request, and spend a full multi-index budget trying to invert a function that is exactly x^2 .

The witness uses the elementary identity

$$a = \sinh^2(1) = \frac{\cosh(2) - 1}{2} = b.$$

Wolfram’s simplifier proves $a - b = 0$, but the package’s separate canonicalization of a and b leaves different expression trees. Its ordinary merger groups by those trees, not by proved equality. The Fourier merger already uses the stronger equality-aware strategy. Thus the defect is localized and repairable without redesigning the coefficient theory.[2, 6]

Three public examples were executed before and after a focused replacement of that grouping step. The repair changes a zero-term result to the exact one-term expression x^2 , changes a product’s remainder degree from 3 to the required 6, and changes an inverse-enumeration failure to the exact answer $\sqrt{y}$. These are native Wolfram observations, not Python simulations. They do *not* constitute a run of the full repository suite.

Two additional contributions matter. First, the real-domain concern already catalogued as C07 now has concrete public witnesses: `ArcSin[2]` bypasses a syntactic ban on `Complex` atoms, and the real-inverse constructor returns $y - \arcsin(2)$ with a nonreal target limit. The coefficients are correct as complex formulas; the defect is that they are accepted under a real-branch contract. Second, a high-precision numerical check can return an unresolved numerical zero for an error that is provably positive. Wolfram correctly retains the zero’s poor accuracy, but the surrounding association has no explicit error-resolution status. This is a diagnostic and usability issue, not evidence that rational interval certification is false.[2, 15, 14]

The practical priority is therefore narrow: repair semantic support grouping; enforce original-function realness at all public real-branch boundaries; and make numerical error resolution explicit. The archive provides an opt-in merge hotfix, a hash-guarded source patcher, regression specifications, a numerical-diagnostic prototype, selected native observations, and independent exact oracles.

The broader comparison should remain fair. Wolfram Language is not limited to Taylor and Puiseux asymptotics: its documentation expressly permits automatically inferred, more exotic scales. The repository’s advantage is the explicit, inspectable machinery for its selected real scales, including exact-weight coefficient access and transported remainder objects, not a universal claim that native functions cannot handle logarithms or irrational powers.[17, 18]

Contents

Executive assessment	1
1 Scope, evidence, and non-duplication	3
1.1 An immutable snapshot	3
1.2 What was inspected and what was executed	3
1.3 The existing reviews form the baseline, not this report's content	4
2 What the repository contributes, and where native Wolfram fits	5
2.1 A useful architecture, not merely a replacement for InverseSeries	5
2.2 A comparison by task and contract	6
2.3 What the native probes actually showed	7
2.4 Where the implementation deserves credit	7
3 A1: semantic weight collisions invalidate the ordinary support invariant	8
3.1 The intended invariant	8
3.2 A small exact witness	8
3.3 A false remainder, not merely untidy output	9
3.4 Why the two-pointer boundary scan misses the worst term	10
3.5 The same defect can create a non-small inverse model	10
3.6 The focused repair and what was verified	10
3.7 A safe repair process	11
3.8 Limits of the repair	11
4 A2: syntactic exactness is not a real-domain proof	12
4.1 The exact/real distinction	12
4.2 The native analytic path can introduce explicit complex coefficients	12
4.3 The real-inverse constructor accepts a nonreal target endpoint	13
4.4 A repair that reuses existing machinery	13
4.5 Why changing the documentation alone is insufficient	14
5 A3: numerical error needs a resolution state	14
5.1 What the current routine does correctly	14
5.2 The executed example	14
5.3 An exact rational proof that the error is nonzero	15
5.4 This is not a counterexample to the asymptotic remainder	16
5.5 A concrete output policy	16
6 Development recommendations arising from the new evidence	16
6.1 Treat semantic support as a checked mathematical object	16
6.2 Add representation-changing metamorphic tests	17
6.3 Move realness proof to the semantic boundary	17
6.4 Make numerical resolution a separate acceptance criterion	17
6.5 A staged integration plan without repeating the existing roadmap	18
7 Validation, use of the artifacts, and conclusions	18
7.1 Using the source repair	18
7.2 Running the supplied checks	19
7.3 What remains unverified	19
7.4 Conclusion	19
A Source coverage and evidence map	20
B Artifact inventory	21
C Primary references	22

1 Scope, evidence, and non-duplication

1.1 An immutable snapshot

All repository findings refer to 921387e5ba1239bfda96e63e64e89bf63d9c41e6. The root standalone file loaded in the native evaluator was 574,894 bytes and exposed 37 public symbols in the package context. The repository identifies this API as version 1.8.0 and requires Wolfram Language 15.0 or later. The modular kernel is the development source; the root file is a generated standalone distribution.[1, 2]

The native environment reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

This is the environment of the observations, not a claim about the newest available Wolfram point release. The current documentation was consulted separately on the audit date. No performance claim is extrapolated from another operating system or from the repository’s historical validation records.

1.2 What was inspected and what was executed

The source inspection concentrated on the ordinary coefficient and precision algebra; forward expansion and object construction; series arithmetic and composition; Fourier coefficients; reciprocal-log calculus; source-coordinate reconstruction; flat-sector operations; inverse branch selection; special-function identities and real-domain contracts; Gamma/Barnes observable powers; exact interval certificates; and numerical inverse checks. Appendix A gives the inspected paths and distinguishes them from features assessed at the documented interface level.

Repository content was read through the GitHub connector. There was no complete checkout in the local build container. The native Wolfram evaluator could load the pinned standalone package using a downloaded file, which permitted the focused examples and the merge transformation to be executed. Some later evaluator or external-download calls failed at the service boundary; they are not counted as package failures. The article and artifact bundle were built locally.

The evidence has four levels:

Native observation. A displayed public input and selected output were obtained from Wolfram Language at the stated version. The machine-readable record is `results/native_observations.js` on; it is a selected transcription, not a full kernel-session export.

Mathematical proof. The false remainder and the positive numerical error are established independently of runtime behavior by explicit identities and inequalities below.

Independent execution. The Python oracles use exact rational arithmetic and high-precision decimal evaluation. They test a model of the boundary algorithm and the mathematical witnesses, not the Wolfram implementation itself.

Proposal. The broader realness integration, numerical wrapper, and expanded test matrix are recommendations. They are not represented as completed, package-wide, natively validated fixes.

The exact arithmetic/model run checked 68 degree-pair cases with 204 family assertions, together with the positive-error oracle. Five Python tests exercised the source patcher’s guards on synthetic fixtures.

Those counts must not be added to the number of native repository tests. The supplied nine-test Wolfram regression collection was not executed as a complete TestReport run.

1.3 The existing reviews form the baseline, not this report's content

The pinned code-review/README.md indexes nine reviews. The maintained development register consolidates 87 numbered findings and their associated design and roadmap proposals. That register was used as the primary de-duplication map, with the relevant earlier discussion inspected where the present evidence sharpens or corrects it. This does not claim a line-by-line rereading of every page of all nine manuscripts.[3, 4, 5]

Item here	Relation to prior work	What is actually added
A1	New root cause; adjacent to C04 and C12	The comparator proves equality, but structural grouping ignores it. A public product has a false remainder; the ordinary multi-index model can acquire a zero gap. A tested grouping repair is supplied.
A2	Sharpens C07	Public constant, analytic-function, and real-inverse examples demonstrate the gap. The distinction between syntactic exactness and original-function realness is made operational.
A3	Sharpens the numerical-evidence/API discussion, including D05	A returned zero is shown to be unresolved, with a strictly positive rational error bracket and a proposed explicit resolution state. The legacy naming of the reference root is not re-reported as new.
Excluded	C01, C02 and the rest of the unchanged register	Dense native export and fractional powers of pure uncertainty have focused fixes in the current snapshot. Previously catalogued resource, refinement, schema, licensing, and extension proposals are not reproduced as new findings.

Two distinctions prevent mistaken issue closure. A1 is not C12's concern about the last branch of an exact-order comparator: the equality required by this witness is already proved. Nor is A1 C04's unit-series frontier-degree rule: the failing product runs through ordinary multiplication and its two-pointer boundary scan. Repairing A1 does not establish that either earlier issue is resolved.[2, 4]

Review 2 describes complete weight grouping before term counting as a strength. That description needs a qualification for ordinary transcendental weights: the intended property is present for identical canonical keys, but does not hold for every equality the comparator can prove. The correction is consequential because the same invariant supports both term-count semantics and remainder proofs.[5, 2]

2 What the repository contributes, and where native Wolfram fits

2.1 A useful architecture, not merely a replacement for InverseSeries

The ordinary representation is a finite jet

$$F(u) = \sum_{\alpha \in A} u^\alpha P_\alpha(L) + O\left(u^p(1 + |L|)^d\right), \quad u \rightarrow 0^+, \quad L = \log u, \quad (1)$$

with exact real weights and polynomial logarithmic coefficients. At a finite endpoint, u is the selected positive distance to that endpoint; at positive or negative infinity it is $1/x$ or $-1/x$. The use of a positive coordinate is an important part of the real-branch interpretation.[2]

Ordinary inversion normalizes a model of the form

$$y - y_0 = cu^p \left(1 + \sum_{j=1}^m u^{\delta_j} B_j(L) \right), \quad c \neq 0, \quad p \neq 0, \quad \delta_j > 0. \quad (2)$$

A uniformizer $z = ((y - y_0)/c)^{1/p}$ separates the leading branch from a unit correction. Individual multi-index coefficients, grouped computations, and residual calculations operate on this structured model. The positivity of each gap is mathematically indispensable: it makes the set of relevant multi-indices finite below a fixed weight. A1 shows why this hypothesis must be established after semantic merging, not inferred from distinct expression keys.[2]

The remaining engines are deliberately different. Fourier coefficients have modes $P_\omega(L)e^{i\omega L}$; flat expansions separate exponential degree from the algebraic precision of each sector; reciprocal-log calculus retains a monomial carrier and an analytic unit in $\pm 1/\log y$; Gamma/Barnes inverses retain exact Lambert cores and inverse-logarithmic coefficient blocks. A common result head provides access and provenance, but does not turn these representations into one unrestricted transseries field.[1, 6, 10, 9, 11]

This separation is often a strength. The envelope arithmetic can transport a sum or product error without forcing it into an unsuitable single scale. Its multiplication rule is the conservative identity

$$(e_1 + O(R_1))(e_2 + O(R_2)) = e_1 e_2 + O(|e_1|R_2 + |e_2|R_1 + R_1 R_2).$$

Its real sin, cos, and absolute-value operations rely on scalar Lipschitz bounds rather than an unjustified Taylor expansion of an arbitrary remainder. Exponentiation requires an absolutely small unknown error, whereas a logarithm requires a relatively small error and a positive approximation. These are meaningful contracts to retain while repairing the shared infrastructure.[7]

2.2 A comparison by task and contract

Task	Native Wolfram Language	Repository's additional role or limitation
Ordinary local reversion	<code>Series</code> and <code>InverseSeries</code> provide compact, integrated series workflows.	More explicit real branch, exact-weight, logarithmic-block, remainder, and provenance machinery for the supported generalized models. Native reversion remains an excellent control on ordinary analytic examples.
Implicit asymptotic solving	<code>AsymptoticSolve</code> accepts scalar equations and systems, specified centers, real-domain restrictions, and automatically inferred scales.	A specialized inverse-model algebra with direct coefficient queries and inspectable finite-model residuals. It is not a general replacement for native systems solving.
Special-function asymptotics	<code>Asymptotic</code> and <code>Series</code> have broad native knowledge; the former also dispatches to other asymptotic tasks.	Domain-aware adapters, retained carriers, explicit errors, and selected exact identities. Some coefficients intentionally come from native routines; this is integration, not independent reimplementations.
Nonrational exponents	Native routines may return useful generalized or exact expressions; their asymptotic facilities are not restricted to rational powers.	A sparse support representation intended to treat arbitrary orderable exact real weights. Its particular semantic-equality obligation is central to A1.
Power/log/exp calculus	Native series arithmetic is integrated into ordinary expressions.	Explicit error transport across the package's selected scales and an envelope fallback. Operation eligibility depends on the representation and realness proof.
Branch selection	Native solving supports centers, domains, directions, and conditions.	Retained source/target information and branch inference for selected <code>InverseFunction</code> expressions. A2 is a hole in the common realness boundary, not a denial that branch checks exist.
Numerical comparison	<code>FindRoot</code> , precision tracking, and exact-number evaluation provide building blocks.	<code>InverseNumericalCheck</code> relates a retained equation to an expansion and its error scale. A3 concerns the resolution of the displayed comparison.
Certified finite-point result	Native numerical precision is not itself an interval theorem.	The repository implements rational interval enclosures and explicit elementary-function tails for a limited certificate route. This is distinct from a proof of every asymptotic remainder.

The native-side statements in this table follow the official references; the repository-side statements follow its README and inspected implementation.[19, 20, 21, 18, 17, 1, 16]

A particularly important comparison boundary is `SeriesData`. Its documented representation stores a coefficient list, minimum and maximum numerator orders, and a denominator defining the exponent lattice. An arbitrary irrational support or a separate logarithmic bound does not have a direct, lossless encoding in those fields. That explains why a separate object is useful; it does not justify overlooking the package’s own support invariant.[\[21\]](#)

2.3 What the native probes actually showed

A polynomial control behaved as expected:

```
AsymptoticSolve[x + x^2 == y, x, {y, 0, 4}, Reals]
(* {{x -> -1-y+y^2-2 y^3+5 y^4},
    {x -> y-y^2+2 y^3-5 y^4}} *)

InverseSeries[Series[x + x^2, {x, 0, 5}]]
(* x - x^2 + 2 x^3 - 5 x^4 + 14 x^5 + 0[x]^6 *)
```

These results confirm both the usefulness of native controls and the need to distinguish selecting a local inverse from finding several branches. The package’s direct inverse call with a specified source endpoint selects a different contract from a native request for all nearby branches.

Two bounded probes of the native solver returned unevaluated inputs in this environment:

```
AsymptoticSolve[x + x^Sqrt[2] == y, x,
  y -> Infinity, Reals, SeriesTermGoal -> 3]
AsymptoticSolve[x + x^2 Log[x] == y, x,
  y -> 0, Reals, SeriesTermGoal -> 3]
```

These are evidence about the exact inputs, options, and kernel tested. They are not a theorem that no change of variables, branch specification, preprocessing, or future native version can solve the problems. The package’s documented specialized scope addresses precisely such models, but a broad superiority benchmark was not performed here.[\[1, 18\]](#)

Other controls demonstrate why careless comparisons are misleading. Native `Series[x^Sqrt[2]+x^Pi, {x, 0, 4}]` returned the exact finite expression, not a failure. Native `Asymptotic[Exp[x+1/x], x->Infinity, SeriesTermGoal->3]` retained the exact exponential expression. Retaining an exact answer is not an incorrect asymptotic result, even when a user would prefer an expanded correction bracket. A native `Erfc` probe returned its familiar exponentially small carrier with algebraic corrections. The delivered comparison script preserves these inputs so later versions can be assessed without changing the test definition.

2.4 Where the implementation deserves credit

The inspected code contains several safeguards that distinguish it from a formula-only experiment. The Fourier path checks reality by conjugate modes and fails rather than silently deleting frequencies. The inverse branch selector distinguishes an inconclusive proof from a rejected branch, and does not equate a bounded candidate search with completeness. Special-function identities retain subdominant exponentials in the supported exact reductions. The reciprocal-log calculus explicitly states the analytic derivative contract it uses.[\[6, 12, 13, 9\]](#)

The certificate code uses exact rational interval operations, explicit exponential and logarithmic tails, and a nonzero derivative bound. The numerical route separately checks a recovered root against the retained source domain and labels the result as numerical evidence. Those distinctions are sound design choices; none of the present findings licenses a claim that every branch proof or interval certificate is unreliable.[16, 15]

3 A1: semantic weight collisions invalidate the ordinary support invariant

Priority: high correctness; native-reproduced. **Novelty:** a new mechanism with a false-bound witness and a focused tested repair.

Source locus: AsymptoticInverse/Kernel/AsymptoticInverse.wl; jetMerge, jetProductBoundaryDegree, pMul, forwardCore; also compare fourierWeightGroups in FourierCoefficients.wl.[2, 6]

3.1 The intended invariant

For an ordinary jet, each semantic weight should occur exactly once, carrying the sum of all its coefficient polynomials. After zero coefficients are removed, the weights should be strictly increasing:

$$\alpha_1 < \alpha_2 < \cdots < \alpha_n, \quad P_{\alpha_i} \neq 0. \quad (3)$$

This is more than a convenient normal form. It makes the first row the true leading block; makes a term goal count complete nonzero blocks; makes a cutoff independent of how an exponent was written; and justifies the linear-time boundary scan used by multiplication.

An exact simplifier is not, in general, a canonical representative of every equivalence class of real expressions. It can prove two expressions equal when asked about their difference, while simplifying each separately to different syntax. The source’s relevant non-symbolic grouping step is

```
groups = GatherBy[
  {canon[#[[1]]], #[[2]]} & /@ terms, First];
```

GatherBy identifies equal keys by expression identity. The later numerical/symbolic order comparator does not retroactively combine those groups. Consequently, a perfectly successful equality proof in the comparator cannot repair an earlier partition by unequal keys.[2]

3.2 A small exact witness

Set

```
a = Sinh[1]^2;
b = (Cosh[2] - 1)/2;
FullSimplify[a - b]
(* 0 *)
```

The native evaluator retained Sinh[1]^2 and (Cosh[2]-1)/2 as different separately simplified forms. Both are positive exact real numbers. Their common value is about 1.3811, so it lies below 2.

The full forward expression

```
AsymptoticExpansion[x^a - x^b + x^2, {x, 0, 3}]
```

retained separate opposite-coefficient rows at the two equal weights. Its finite expression still denotes x^2 on $x > 0$, so this first observation alone is not a false numerical formula. It exposes a broken representation invariant.

The one-term request makes the user-visible consequence immediate:

```
s = AsymptoticExpansion[x^a - x^b + x^2,
  {x, 0}, SeriesTermGoal -> 1];
{Normal[s], s["Remainder"], s["ReturnedTermCount"]}
(* {0, PowerLogRemainder[x,b,0], 0} *)
```

The mathematically complete answer is the single exact block x^2 . Instead, the unmerged equal weights are counted when choosing a cutoff; the exclusive cutoff can then exclude both of them. This is not the intended convention that a term goal counts complete nonzero weight blocks. The zero expression plus the coarse bound is not by itself a false big- O assertion, but it does not fulfill the requested term goal.

3.3 A false remainder, not merely untidy output

Consider the exact function

$$F(x) = 1 + x^a + x^b (\log x)^3 + x^{2a}, \quad x > 0. \quad (4)$$

The public sequence

```
s = AsymptoticExpansion[
  1 + x^a + x^b Log[x]^3 + x^(2 a),
  {x, 0, 2 a}];
t = SeriesMultiply[s, s];
{Normal[t], t["Remainder"]}
(* {1 + 2 x^a + 2 x^b Log[x]^3,
  PowerLogRemainder[x, 2 a, 3]} *)
```

was reproduced in the native kernel. The input series has a valid remainder $O(x^{2a})$. The output, however, claims a logarithmic degree that is too small.

Proposition 3.1. *For the exact function in (4), the reported product error is not $O(x^{2a}(1 + |\log x|)^3)$. Its leading logarithmic degree at weight $2a$ is 6.*

Proof. Put $L = \log x$ and $t = x^a$. Since $a = b$,

$$F = 1 + t(1 + L^3) + t^2.$$

Subtracting the reported finite product gives the exact identity

$$F^2 - (1 + 2t(1 + L^3)) = t^2((1 + L^3)^2 + 2) + 2t^3(1 + L^3) + t^4. \quad (5)$$

As $x \rightarrow 0^+$, $t \rightarrow 0$ and $L \rightarrow -\infty$. Dividing by $x^{2a}(1 + |L|)^3$ gives a quantity asymptotic to $|L|^3$, which is unbounded. Dividing by $x^{2a}(1 + |L|)^6$ instead gives a quantity tending to 1. Therefore degree 3 is false and degree 6 is valid. $\square$

The counterexample uses an exact admissible representative of the input remainder class. It therefore disproves the asserted transport for that class; it does not rely on guessing the hidden constant in a big- O symbol or numerically sampling close enough to an endpoint.

3.4 Why the two-pointer boundary scan misses the worst term

`jetProductBoundaryDegree` walks one support from the beginning and the other from the end. When the two weights sum to the boundary, it records the sum of their coefficient degrees, then advances both pointers. That is a valid linear-time enumeration of boundary pairs when both supports satisfy (3). With duplicate semantic weights, moving both pointers can skip other pairs at the same sum.[2]

For the witness, the two equal-weight rows have degrees 0 and 3. The scan encounters cross-pairs of degree 3 and misses the degree-6 product of the two cubic-log rows. This explains both the exact observed degree and why a correct error formula in `pMul` is insufficient without a correct support representation.

The severity is not bounded by three missing logarithms. More generally, take noncancelling coefficient polynomials L^p and L^q in separate equal-weight rows, with $0 \leq p < q$. The duplicate-support scan records $p + q$, while the merged coefficient has degree q and its square has degree $2q$. The loss is $q - p$, which can be arbitrarily large. The independent model checks cover 68 such degree pairs. This generalization is a mathematical/source-algorithm analysis; only the displayed $p = 0, q = 3$ public product was run in Wolfram.

3.5 The same defect can create a non-small inverse model

The exact identity $x^a - x^b + x^2 = x^2$ also yields

```
AsymptoticInverse[x^a - x^b + x^2,
  {x, 0}, {y, 2}]
(* Failure["ResourceLimit", <|
  "MessageTemplate" ->
    "Multi-index enumeration exceeded MaxTerms.",
  "MaxTerms" -> 20000|>] *)
```

The selected real inverse is simply $\sqrt{y}$. Retaining two equal leading weights can create a correction gap equal to zero. The multi-index enumeration is then being asked to work with a model outside the strictly positive-gap hypothesis in (2). Raising `MaxTerms` is not a remedy for this example.

This is a concrete performance failure caused by a correctness invariant, not a benchmark showing that one general inversion method is faster than another. It should be triaged before tuning enumeration constants or caches.

3.6 The focused repair and what was verified

The tested transformation replaces the structural partition with sorting followed by proved-equality grouping:

```
groups = Split[
  Sort[{canon#[[1]], #[[2]]} & /@ terms,
    less[#1[[1]], #2[[1]]] &],
```

```
equal[#1[[1]], #2[[1]]] &;
```

The existing coefficient summation and zero elimination then operate on complete semantic classes. The Fourier implementation already uses the same pattern for its weights and frequencies, so this recommendation aligns the ordinary engine with a precedent in the repository rather than introducing a novel comparison convention.[6]

The native rerun after this transformation produced:

Example	Before	After
One-term cancellation	0, zero returned terms	x^2 , zero remainder, one term
Product boundary	degree 3	degree 6
Inverse of exact x^2	budget failure	$\sqrt{y}$, zero remainder

This validates the transformation against the three focused public examples. It does not validate every caller, the complete hotfix installation/removal wrapper, or all symbolic-weight behavior.

3.7 A safe repair process

The delivered runtime hotfix is opt-in and session-local. It saves the original downvalues, requires exactly one expected structural-grouping anchor, and refuses to undo unrelated later changes. The source patcher is stricter: it accepts only the audited modular source blob

```
e01210c050dc923215c08d808db52fab140a7c4a
```

and the unique exact source anchor. It defaults to a dry run, preserves a backup when applying the patch, and does not modify the generated root distribution.

A production integration should also assert the positive-gap invariant at inverse-model construction. That assertion is not a substitute for merging: it would turn the example into an explanatory failure rather than the correct answer. Its purpose is to prevent another support defect from reaching an enumeration routine whose finiteness proof assumes positive gaps.

For performance, a two-stage implementation is reasonable: first combine identical canonical keys cheaply; then sort the remaining representatives, coalesce adjacent proved-equal weights, and combine their coefficient sums. If there are n raw rows and m structural groups, this leaves approximately $O(n)$ structural collection plus $O(m \log m)$ comparison work. This is a complexity argument, not a measured speedup. The minimal patch favors clarity and consistency with the Fourier implementation; a fast-path optimization should follow profiling and invariant tests.

3.8 Limits of the repair

Equality here must be proved, not inferred from a numerical tolerance. Distinct exact exponents can be arbitrarily close. Nor should a failed proof be converted into equality or a guessed strict order. The broader comparator concern in the existing register remains separate. The runtime messages also show repeated precision-limit warnings while comparing equivalent forms; the merge repair removes their repeated downstream consequences, but does not itself redesign that comparison procedure.

Finally, this patch does not fix the earlier nonlinear unit-frontier degree issue, native-series export policy, arbitrary restored-object validation, or all possible composite-scale operations. Its claim is deliberately specific: ordinary support classes must reflect equality that the implementation already knows how to prove.

4 A2: syntactic exactness is not a real-domain proof

Priority: high contract integrity; native-reproduced. **Novelty:** concrete public witnesses and a sharper repair boundary for the earlier C07 concern.

Source locus: `AsymptoticInverse/Kernel/AsymptoticInverse.wl`; `exactQ`, `validateInput`, `pConst`, `fwDA`, `analytic`, `inverseobjectconstruction`; `compare specialFunctionRealDomain` in `SpecialFunctionRealDomain.wl`.[\[2, 14\]](#)

4.1 The exact/real distinction

The shared syntactic test is essentially

```
exactQ[e_] := FreeQ[e, _Real | _Complex]
```

and the associated validation message says that exact real input is required. This test rejects machine or arbitrary-precision real atoms and explicit complex atoms. It does not establish that an exact expression denotes a real number. An unevaluated special-function value can be nonreal without containing either forbidden head.[\[2\]](#)

The contrast is visible in public calls:

```
AsymptoticExpansion[x + I, {x, 0, 3}]
(* Failure["InexactInput", ...] *)

s = AsymptoticExpansion[x + ArcSin[2], {x, 0, 3}];
{Normal[s], s["Remainder"], s["TargetDomain"]}
(* {x + ArcSin[2], 0, x > 0} *)
```

The first rejection is not evidence that the second expression is real. Under the principal branch,

$$\arcsin(2) = \frac{\pi}{2} - i \log(2 + \sqrt{3}), \quad \log(2 + \sqrt{3}) > 0. \quad (6)$$

For example, the sine of the right-hand side is $\cosh(\log(2 + \sqrt{3})) = 2$, with the principal-branch sign consistent with Wolfram's convention. The official `ArcSin` reference documents complex values and branch behavior; it is not a real-valued function on all real arguments.[\[22\]](#)

4.2 The native analytic path can introduce explicit complex coefficients

The problem is not limited to a constant that remains visibly unevaluated:

```
AsymptoticExpansion[ArcSin[2 + x], {x, 0, 3}]
(* ArcSin[2] - I x/Sqrt[3] + I x^2/(3 Sqrt[3])
   + PowerLogRemainder[x,3,0] *)
```

The output's recorded target condition was $x > 0$. The Taylor coefficients are mathematically correct for the complex principal branch, but the original function is nonreal throughout that positive neighborhood. Initial input validation cannot catch complex coefficients that are subsequently produced by a native analytic expansion.

This is why inserting another test for a literal `I` at the front door is not an adequate repair. It would still miss encoded nonreal constants, and it would not establish reality of a discarded tail or of the original function.

4.3 The real-inverse constructor accepts a nonreal target endpoint

The stronger witness is

```
s = AsymptoticInverse[x + ArcSin[2],
  {x, 0}, {y, 3}];
{Normal[s], s["Remainder"], s["Limit"]}
(* {y - ArcSin[2], 0, ArcSin[2]} *)
```

Querying `TargetDomain` on this returned ordinary inverse object gave a missing-key result. That missing key is not the primary finding: even a positive-side condition cannot make a nonreal endpoint a real inverse endpoint.

For real x , $x + \arcsin(2)$ has nonzero imaginary part by (6); consequently it does not equal any real y . The returned expression is an exact complex inverse formula, but it is not the inverse of a map between real neighborhoods with the advertised source and target interpretation.

It would be inaccurate to call this an erroneous Taylor coefficient or a counterexample to a complex inversion identity. The failing promise is the real-domain contract. That distinction matters when deciding whether to reject the expression or deliberately introduce a separate complex-domain API.

4.4 A repair that reuses existing machinery

The repository already contains a sufficient-real-domain prover. It first attempts a symbolic proof of realness on the positive local coordinate, then applies principal real-domain contracts for recognized heads. Its `ArcSin` rule requires an argument in $[-1, 1]$, and its constant case requires a proof of membership in the reals. It returns inconclusive failure rather than assuming a real domain.^[14]

The proposed integration has three layers. At the public real-domain boundary, strip and retain user conditions, normalize the local chart, and require an eventual realness proof for the original expression under the effective assumptions. At native coefficient import or analytic expansion, verify that the resulting ordinary coefficient polynomials are real under the retained conditions. At real-inverse model construction, require a real finite target limit and real leading/correction data, not merely syntactic exactness of the source input.

These checks address different obligations. A real finite coefficient list does not prove that an unrepresented exponentially small contribution is real; conversely, a sound original-function proof does not excuse an incorrectly imported coefficient. A coefficient check should be an invariant check in addition to, not instead of, the original-function domain proof.

No global realness patch is claimed in this archive. The existing domain prover has deliberately bounded sufficient rules and cannot necessarily prove every valid real expression. Requiring it unconditionally without fallback analysis could reject previously supported real cases. The safe result for an unresolved

real-branch obligation is an explanatory failure or an explicitly conditional result, not a silently assumed real germ. The supplied tests specify the three necessary rejections and preserve the explicit-complex rejection as a control.

4.5 Why changing the documentation alone is insufficient

Some package arithmetic is genuinely real-domain arithmetic. The composite envelope route's global 1-Lipschitz bounds for $\sin$ and $\cos$ are valid on the real line, not on the entire complex plane. Fractional-power branches and positivity checks likewise depend on the real interpretation. Thus changing a usage string to say that some complex expressions happen to be accepted would not supply the missing contracts.[7]

Existing downstream guards do reject many nonreal operands, and the interval certificate route supports a narrower set of expressions. The present examples do not show that a false interval certificate was issued, nor that every arithmetic path propagates the nonreal object unsafely. They show that a real-domain invariant must be established consistently before such paths decide what they may assume.

5 A3: numerical error needs a resolution state

Priority: medium diagnostic reliability. Novelty: an executed precision-floor example with a strictly positive exact error bracket.

Source locus: `AsymptoticInverse/Kernel/NumericalInverseChecks.wl;numericalInverseEvidence.`[15]

5.1 What the current routine does correctly

The numerical checker substitutes an exact target before numerical evaluation, uses a higher working precision internally than requested, solves the retained original equation, checks the source side and retained domain, and reports both forward and root residuals. Its evidence text explicitly says that the result is numerical comparison rather than an interval certificate. The `ExactInverse` field is a documented legacy alias for the reference root; this audit does not re-report that name as a new defect.[15]

The remaining issue is that a fixed guard precision is not a guarantee that the difference of two nearly equal answers is resolved. Precision sufficient for a root value can be grossly insufficient for its truncation error. Native arbitrary-precision tracking does preserve this distinction, but the returned association does not elevate it to a clear success, unresolved, or insufficient-input status.[23, 24]

5.2 The executed example

Let $g(y)$ be the positive small root of $x + x^2 = y$. Its exact formula and convergent expansion near zero are

$$g(y) = \frac{\sqrt{1+4y}-1}{2} = \sum_{n \geq 1} (-1)^{n-1} C_{n-1} y^n, \quad C_n = \frac{1}{n+1} \binom{2n}{n}. \quad (7)$$

Write S_{19} for the truncation through y^{19} . The native call was


```
s = AsymptoticInverse[x + x^2, {x, 0}, {y, 20}];
c = InverseNumericalCheck[s, 1/1000,
  WorkingPrecision -> 20];
```

Selected returned values were

```
"Error"      -> 0.'32.69940364882006
"RemainderScale" -> 1.'20.*^-60
"Ratio"      -> 0.'-27.300596351179944
"RootResidual" -> 0.'32.99956678807954
```

The true absolute error is approximately

$$|S_{19}(10^{-3}) - g(10^{-3})| = 1.7607234447062309131509951812539731 \dots \times 10^{-51}. \quad (8)$$

Therefore the returned error is *unresolved*, not an exact zero. The ratio's negative accuracy makes that especially clear in full Wolfram input form. A client that drops the accuracy annotation, or a reader who sees a rounded zero in a table, loses this crucial distinction.

A less extreme run with cutoff 12, the same target, and the same requested working precision returned an error around 5.85787×10^{-32} , but only about 1.467 digits of reported precision in that error and ratio. This gives a useful transition case: the issue is not only a binary zero/nonzero event, but also whether the comparison meets a meaningful error-digit requirement.

5.3 An exact rational proof that the error is nonzero

Numerical subtraction is unnecessary for the decisive conclusion. At the rational target $y = 1/1000$, $S = S_{19}(y)$ is rational. Compute the rational forward residual

$$R = S + S^2 - y.$$

The delivered oracle computes R exactly and verifies $R > 0$. Since $g + g^2 = y$,

$$R = (S - g)(1 + S + g). \quad (9)$$

The positive root satisfies $0 < g < y$. It follows that

$$0 < \frac{R}{1 + S + y} < S - g < \frac{R}{1 + S}. \quad (10)$$

Both endpoints are explicit positive rational numbers. This proves the nonzero error without trusting a high-precision reference-root subtraction. The exact fractions and decimal evaluations are retained in `results/independent_oracles.json`.

For a stable decimal evaluation of (8), the oracle uses

$$g = \frac{2y}{1 + \sqrt{1 + 4y}}, \quad S - g = \frac{R}{1 + S + g}.$$

The rational residual is formed before conversion to decimal. This avoids the catastrophic loss that would arise from subtracting two independently rounded values of S and g .

5.4 This is not a counterexample to the asymptotic remainder

The package's remainder scale for the cutoff-20 expansion is $y^{20} = 10^{-60}$ at this target. The actual error divided by that scale is about 1.76×10^9 . A large ratio does not refute an asymptotic big-O bound: the leading omitted coefficient is a large Catalan number, and the bound does not assert a constant of one. The relevant error here is loss of numerical resolution, not the size of the big-O coefficient. Nor does a numerical zero with finite accuracy assert exact equality in Wolfram's numeric model. The improvement is to make the package's *diagnostic contract* say plainly when it cannot measure the requested error.

5.5 A concrete output policy

A useful result should retain the raw numeric values while adding an explicit resolution state. For example, "NumericallyResolved" can mean that the nonzero error and its normalized ratio both carry at least a requested number of significant digits. "UnresolvedAtWorkingPrecision" can mean that an error difference is numerically zero or has too few reliable digits. A separate state is needed when the target itself has insufficient supplied precision for refinement.

The source equation and asymptotic error scale can guide an adaptive precision request. A rough planning estimate compares the magnitude of the approximation with the magnitude of the error scale, then adds the desired error digits and a guard. It is only a planning heuristic: an asymptotic error scale has an unspecified constant, and a nonlinear equation can be ill-conditioned. The acceptance check must examine the resolved difference or a certified enclosure, rather than trusting the planned working precision.

The delivered ResolvedInverseNumericalCheck prototype repeats the existing checker at increasing precision and accepts only a positive error and ratio with enough reported digits. It never pads an inexact target with invented precision. It returns a structured failure when the difference remains unresolved. This wrapper is intentionally conservative: an exactly correct approximation can remain a numerical zero at every precision and should then be handled by a separate symbolic or interval argument.

The wrapper was not executed against the package in this audit. The underlying failure examples and the exact oracle were executed. Production integration should also compare successive reference computations, consider conditioning, and preserve the distinction between resolved numerical evidence and a certificate. The prototype's purpose is to make the proposed state transition concrete, not to imply that arbitrary-precision tracking alone proves an error bound.

6 Development recommendations arising from the new evidence

6.1 Treat semantic support as a checked mathematical object

The immediate repair should be followed by a small support contract shared by ordinary jets and their consumers. The contract is not a large new public schema: it is the assertion that weights are ordered, proved-distinct after merging, and attached to nonzero complete coefficient blocks. Constructors and transformations should establish it before a consumer relies on it.

In particular, leading-term extraction, multiplicative frontier scans, inverse gap generation, and term counting should all consume the same normalized support. They should not each silently assume that another path performed the required merge. A debugging or validation build can check adjacent

weights and the positivity of normalized gaps. Those checks directly target the demonstrated defect and are narrower than a generic proposal for a new proof framework.

The Fourier code is a useful internal reference implementation for the equality-aware partition. A shared helper could prevent the ordinary and Fourier paths from diverging again, provided that its contract distinguishes symbolic assumption-dependent weights from fully ordered numeric weights. Sharing an implementation must not accidentally apply an unconditional numeric ordering to a symbolic-depth expansion.[2, 6]

6.2 Add representation-changing metamorphic tests

The package should give equivalent results when an exact exponent is replaced by a provably equal expression. This supplies a test oracle without needing a closed form for every generalized inverse. Start from a supported model, replace selected exponents with equivalent forms, and compare normalized blocks, complete term counts, remainder bounds, and inverse residuals.

The tests should deliberately cover an equality that RootReduce cannot settle merely as an algebraic-number normalization. Hyperbolic double-angle identities provide one family; other proved transcendental identities can be added only after their exact equality is established in the test. Nearby numerical values must never be treated as equivalent test inputs.

The most valuable combinations are cancellation before a term goal, equal-weight terms with different logarithmic degrees, products whose worst boundary pair is not the pair visited by the old scan, and inverse leading blocks whose cancellation changes the leading power. Permuting input order and changing parenthesization should not alter these mathematical conclusions. The supplied examples establish the first regression family; a broader deterministic fuzzer is a development proposal, not a claimed completed test corpus.

6.3 Move realness proof to the semantic boundary

A2 shows that separate syntactic bans and specialized domain checks can leave a gap between dispatch paths. The recommended boundary is the original function on the selected deleted real neighborhood, under the conditions retained in the result. That proof should precede any arithmetic or branch claim that relies on realness.

The existing sufficient-domain machinery provides a starting point. Integration should expose whether the proof succeeded, failed, or was inconclusive, but the important addition here is not another generic capability flag: it is a testable obligation that rejects $\text{ArcSin}[2]+x$ consistently with $I+x$ and rejects a nonreal target endpoint in a real inverse. This is a concrete acceptance criterion for closing the earlier C07 item.

6.4 Make numerical resolution a separate acceptance criterion

A numerical checker should not call a comparison resolved merely because FindRoot returned a number and the root passed a domain predicate. That establishes neither the accuracy of the difference nor that of the ratio. The acceptance criterion should be expressed in error digits, a reference enclosure, or an explicitly selected weaker diagnostic mode.

This improvement is separate from increasing the number of retained asymptotic terms. In the executed example, retaining more terms makes the approximation better while making a fixed-precision

comparison less informative. A good interface should explain that outcome rather than making an apparent zero ratio look like stronger evidence.

6.5 A staged integration plan without repeating the existing roadmap

The first stage is a support-normalization change with focused tests and a positive-gap assertion. Its release criterion is that the three public A1 examples pass, ordinary analytic controls still agree with native reversion, and the existing full suite is run on the supported configurations. The audit does not claim that last step has already happened.

The second stage is a real-domain boundary change, with rejection tests and valid-real controls drawn from the ordinary and specialized paths. Its criterion is not simply that the three nonreal examples fail: ordinary real compositions and conditional source domains must retain their intended support.

The third stage is diagnostic resolution reporting, with tests spanning a clearly resolved error, a low-digit error, an unresolved zero, insufficient input precision, and an exactly zero symbolic error. The rational quadratic oracle is small enough to remain an independent reference across implementation changes.

The prior register already contains proposals for integration, complex sectors, broader coefficient fields, multivariate systems, automatic certificates, deployment, and generalized resource budgets. Repeating that roadmap would not add value here. The present recommendations are deliberately attached to newly demonstrated obligations and their acceptance tests.^[4]

7 Validation, use of the artifacts, and conclusions

7.1 Using the source repair

The Python patcher reads a local checkout and defaults to a dry run:

```
python code/apply_semantic_weight_patch.py /path/to/Asymptotic
python code/apply_semantic_weight_patch.py /path/to/Asymptotic --apply
```

It refuses a changed source blob or an unexpected number of anchors. This is preferable to automatically applying a source-level edit to an arbitrary future version. The patch modifies the modular kernel only. Load that kernel for testing; regenerate the standalone distribution through the repository's own build workflow before distributing a new standalone release.

The session-local alternative leaves the checkout unchanged:

```
Get["/path/to/Asymptotic/AsymptoticInverse.wl"];
Get["code/semantic_weight_hotfix.wl"];
AsymptoticAudit'InstallSemanticWeightFix[]
(* Run focused examples. *)
AsymptoticAudit'RemoveSemanticWeightFix[]
```

The package should be loaded from the pinned version for reproducing this audit. A later version may already contain a different repair and should be reassessed rather than forced through this patch.

7.2 Running the supplied checks

The independent checks require only Python’s standard library:

```
python tests/independent_oracles.py
python tests/test_patch_tool.py
```

The Wolfram regression runner reads the modular kernel from an existing checkout:

```
wolframscript -file tests/run_audit.wls /path/to/Asymptotic
wolframscript -file tests/run_audit.wls /path/to/Asymptotic --semantic-fix
```

The real-domain rejection tests intentionally remain failing after the semantic merge fix; that fix does not address A2. The runner checks that the expected number of tests actually ran and returns a nonzero exit status on failures. It is a small audit runner, not a replacement for the repository’s own suite.

The built-in comparison script uses bounded probes and records expressions rather than reducing every unevaluated answer to a universal unsupported-feature claim. The native numerical wrapper is a prototype with the limitations stated in Section 5. Review its policy before integrating it into a public API.

7.3 What remains unverified

No full native regression suite, complete cross-platform test matrix, or broad performance benchmark was run. The source patcher was tested on synthetic fixtures rather than applied to a full local checkout. The runtime grouping transformation itself was tested on the three principal public examples; the additional installation and restoration guards in the delivered helper were not separately exercised in a native package session.

The report also does not claim to have proved the entire repository article, every special-function adapter, or the absence of other defects. Broadly documented features were compared at their stated contract level, and the direct source coverage is listed in the appendix. These boundaries should remain attached to the artifacts when the findings are transferred into the existing issue register.

7.4 Conclusion

The most useful development action is not another expansion family. It is to make the representation obey the mathematical assumptions already used by its algorithms. A sparse jet is a sum over semantic weights, not a list of expression-tree keys. Once that distinction is enforced, the demonstrated incorrect remainder and the zero-gap inverse failure disappear under the same small repair.

The real-domain and numerical-resolution examples point to the same engineering principle: a representation should record what has actually been established. Absence of a `Complex` atom does not establish a real function; a returned approximate zero does not establish a zero error; and a successful local patch test does not establish a fully validated release. The package already contains good mechanisms for many of these distinctions. The recommended work is to apply them consistently at the shared boundaries, with the new examples as permanent regressions.

A Source coverage and evidence map

The following table records the focus of direct inspection. “Selected paths” means relevant functions or contiguous portions, not every line of the named module.

Path under AsymptoticInverse/Kernel/	Inspection focus
AsymptoticInverse.wl	Exactness, canonicalization, comparison, ordinary jet/precision primitives, forward construction, object properties, residual/numerical dispatch, coefficient access, module loading. Selected paths.
SeriesOperations.wl	Representation conversion, arithmetic, unary observables, composition, differentiation contracts. Selected paths.
SeriesEnvelopeArithmetic.wl	Composite error bounds, target approach checks, exact scalar operands, powers, logarithm/exponential and Lipschitz observables.
FourierCoefficients.wl	Semantic mode grouping, realness, budgets, coefficients, construction, residuals.
ReciprocalLogOperations.wl	Analytic unit conversion, composition, derivative contract and exact carrier treatment.
SourceCoordinates.wl	Logarithmic/exponential source charts, exact reconstruction and error amplification checks. Selected paths.
FlatSectorOperations.wl	Sector/inner precision distinction, alignment, products, polynomial observables and derivative transport. Selected paths.
InverseFunctionBranches.wl	Candidate completeness, real domains, eventual conditions, sign and local monotonicity validation. Selected paths.
SpecialFunctionIdentities.wl	Finite exact reductions and retention of subdominant exponential terms.
SpecialFunctionRealDomain.wl	Original-function realness proof and elementary/special-head domain rules. Selected paths.
GammaInverseOperations.wl	Observable powers and inherited precision caps during source replay.
InverseCertificates.wl	Rational interval arithmetic, elementary enclosures, derivative exclusion of zero, branch/domain checks and refinement.
NumericalInverseChecks.wl	Root seeding, precision, source-domain checks, error/ratio computation and evidence labels.

Other engine families and deployment features were assessed primarily through the README, public interfaces, imports, and maintained review register; their entire implementations were not reread. The direct review of prior manuscripts was targeted, while the consolidated register supplied the cross-review issue map. This is why no blanket statement that every repository line was audited appears in this report.

B Artifact inventory

Artifact	Purpose / validation status
article/asymptotic-incremental-audit.tex	This report and compiled document.
Companion asymptotic-incremental-audit.pdf	
code/semantic_weight_hotfix.wl	Opt-in runtime repair. The underlying grouping transformation was tested; helper guards were not fully native-tested.
code/apply_semantic_weight_patch.py	Hash-guarded modular source patcher; five synthetic-fixture unit tests passed.
code/reliable_numerical_check.wl	Adaptive error-resolution prototype, not package-tested or certified.
tests/semantic_weight_regressions.wlt	Three principal desired invariants plus ordinary controls.
tests/real_domain_regressions.wlt	Desired rejections for A2 and an explicit-complex control.
tests/run_audit.wls	Local native runner; deliberately distinguishes failure from a zero-test run.
tests/builtin_comparison.wls	Repeatable, bounded native comparison inputs.
tests/independent_oracles.py	Exact rational error bracket, duplicate-support boundary model and decimal growth data.
results/native_observations.json	Selected native input/output transcription and explicit execution scope.
results/independent_oracles.json	Executed mathematical/model checks and exact fractions.
results/remainder_growth.csv	Evaluation of the analytically derived bad-bound and valid-bound ratios.
README.md, SOURCES.md, manifest.json	Usage, source map, pinned identity and evidence boundaries.

C Primary references

All repository URLs below are pinned to the audited commit. Documentation pages were consulted on 9 September 2026. The discussion of older review findings is attribution and delta analysis, not a claim that the older manuscripts describe the current implementation unchanged.

References

- [1] V. Reshetnikov, *Asymptotic: README*, audited snapshot.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/README.md>
- [2] *AsymptoticInverse/Kernel/AsymptoticInverse.wl*. Primary source for ordinary weights, precision algebra, and public construction.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/AsymptoticInverse.wl>
- [3] *code-review/README.md*. Index of the nine earlier reviews.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/README.md>
- [4] *docs/development/CODE_REVIEW_STATUS.md*. Maintained consolidation and implementation status.
https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/docs/development/CODE_REVIEW_STATUS.md
- [5] *Technical review and comparison with Wolfram Language*, review 2, earlier snapshot discussed in the manuscript.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-2/article/asymptotic-review.tex>
- [6] *FourierCoefficients.wl*, particularly `fourierWeightGroups` and the coefficient/error algebra.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FourierCoefficients.wl>
- [7] *SeriesEnvelopeArithmetic.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl>
- [8] *SeriesOperations.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesOperations.wl>
- [9] *ReciprocalLogOperations.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/ReciprocalLogOperations.wl>
- [10] *FlatSectorOperations.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FlatSectorOperations.wl>

- [11] *GammaInverseOperations.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverseOperations.wl>
- [12] *InverseFunctionBranches.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/InverseFunctionBranches.wl>
- [13] *SpecialFunctionIdentities.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SpecialFunctionIdentities.wl>
- [14] *SpecialFunctionRealDomain.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SpecialFunctionRealDomain.wl>
- [15] *NumericalInverseChecks.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/NumericalInverseChecks.wl>
- [16] *InverseCertificates.wl*.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/InverseCertificates.wl>
- [17] Wolfram Research, *Asymptotic*.
<https://reference.wolfram.com/language/ref/Asymptotic.html>
- [18] Wolfram Research, *AsymptoticSolve*.
<https://reference.wolfram.com/language/ref/AsymptoticSolve.html>
- [19] Wolfram Research, *Series*.
<https://reference.wolfram.com/language/ref/Series.html>
- [20] Wolfram Research, *InverseSeries*.
<https://reference.wolfram.com/language/ref/InverseSeries.html>
- [21] Wolfram Research, *SeriesData*.
<https://reference.wolfram.com/language/ref/SeriesData.html>
- [22] Wolfram Research, *ArcSin*.
<https://reference.wolfram.com/language/ref/ArcSin.html>
- [23] Wolfram Research, *WorkingPrecision*.
<https://reference.wolfram.com/language/ref/WorkingPrecision.html>
- [24] Wolfram Research, *Accuracy*.
<https://reference.wolfram.com/language/ref/Accuracy.html>